CNN Basics

Structured Notes

Why Fully Connected Networks Fail for Images, Images as Tensors, and Convolution Intuition

Contents

1 Why Fully Connected Networks Fail for Images **2**
1.1 Flattening and Its Implications . . . 2
1.2 Parameter Explosion . . . 2
1.2.1 Scaling to Realistic Image Sizes . . . 3
1.3 Loss of Spatial Structure . . . 3
1.4 What We Need . . . 4

2 Images as Tensors **4**
2.1 What is a Pixel? . . . 4
2.2 Image as a 3D Tensor . . . 4
2.2.1 Color Channels . . . 5
2.3 PyTorch Tensor Convention . . . 5
2.4 Why Channel Before Height and Width? . . . 6
2.5 Role of the Batch Dimension . . . 6

3 Convolution — Intuition and Mechanics **6**
3.1 The Sliding Window Idea . . . 7
3.2 Convolution Example . . . 7
3.3 Kernel as a Pattern Detector . . . 7
3.4 Convolution with Multi-Channel Images . . . 8
3.5 Output Size Reduction . . . 8
3.6 Kernel Size Tradeoffs . . . 9
3.7 Padding . . . 9

4 Key Concepts from Deep Learning Context **9**
4.1 Deep Learning and Data Requirements . . . 9
4.2 Curse of Dimensionality and Deep Learning . . . 10

5 Summary **10**

1 Why Fully Connected Networks Fail for Images

Before applying Convolutional Neural Networks (CNNs), it is essential to understand why standard Artificial Neural Networks (ANNs) with fully connected layers are not suitable for image data.

Fully Connected Network

A **fully connected network** (also called a dense network or ANN) is a neural network where every neuron in one layer is connected to every neuron in the next layer. The pipeline is: **Flatten** → **Linear** → **Activation** → **Repeat**.

The ANN pipeline requires input data to be a 1D vector. For tabular data with features x_1, x_2, x_3, this works well because the order of features does not carry spatial meaning. However, images have an inherently different structure.

1.1 Flattening and Its Implications

For a data matrix of shape $N \times D$ (where N is the number of data points and D is the number of features), each row is fed as a vector into the neurons. A matrix of shape 4×3 gets fed directly — each row has 3 features, each neuron has 3 weights.

For a 2D matrix of shape 4×4, flattening converts it into a 16×1 column vector. This is necessary because neurons in an ANN accept 1D input.

Key Observation

For tabular data, the order of features in the flattened vector does not matter — we only need the features to undergo linear transformation. However, for images, spatial ordering carries critical information that is destroyed by flattening.

1.2 Parameter Explosion

Consider a small RGB image:

- Height = 32, Width = 32, Channels = 3

Total input features after flattening:

$$32 \times 32 \times 3 = 3{,}072$$

Connecting to just 100 hidden neurons:

$$\text{Total weight parameters} = 3{,}072 \times 100 = 307{,}200$$

Plus 100 bias terms. That is already **∼300K parameters in just one layer**.

Parameter Explosion Demonstration

```
H, W, C = 32, 32, 3
hidden_neurons = 100

input_features = H * W * C
params = input_features * hidden_neurons + hidden_neurons

print("Input features:", input_features)          # 3072
```

```
print("Parameters in first FC layer:", params)     # 307300
```

Listing 1: Computing parameters for a fully connected layer

1.2.1 Scaling to Realistic Image Sizes

A typical 12-megapixel camera image has approximately $3{,}000 \times 4{,}000$ pixels. With 3 RGB channels:

$$3{,}000 \times 4{,}000 \times 3 = 36{,}000{,}000 \text{ input features}$$

Connecting to just 100 neurons:

$$36{,}000{,}000 \times 100 = 3{,}600{,}000{,}000 \text{ parameters (3.6 billion)}$$

Even a modest 128×128 RGB image produces 49,152 input features, leading to ∼5 million parameters in a single layer. This is why large language models and deep learning models have billions of parameters — the data dimensions drive parameter counts.

Image Size	Input Features	Params (100 neurons)
$32 \times 32 \times 3$	3,072	307,300
$128 \times 128 \times 3$	49,152	4,915,300
$3000 \times 4000 \times 3$	36,000,000	3,600,000,100

Large parameter counts lead to:

- High memory consumption
- Slower computation
- Higher risk of overfitting
- Difficult training

1.3 Loss of Spatial Structure

When an image is flattened:

$$(H, W, C) \to (H \times W \times C)$$

the spatial relationships between pixels are destroyed.

Spatial Information Loss

Two neighboring pixels in the 2D grid may become far apart in the 1D flattened vector. The ANN has no way to know:

- Which pixels were neighbors
- Which pixels formed edges
- Which pixels formed shapes

Consider a face image represented as a matrix. Specific regions contain eyes, nose, mouth, etc. When flattened, features from the eye region may become interleaved with features from the nose or hair region. The model cannot reconstruct the spatial arrangement, making it impossible to detect spatial patterns like “eyes should be above the nose.”

Furthermore, every image may flatten differently in terms of which spatial features end up adjacent, making consistent learning impossible.

Interview Question

Why can we not directly use ANN for image classification instead of CNN?

Two key reasons:

1. **Parameter explosion**: The number of parameters grows rapidly with image size, leading to high computational cost and overfitting risk.
2. **Loss of spatial structure**: Flattening destroys the 2D spatial relationships between pixels that are essential for recognizing patterns in images.

1.4 What We Need

Images contain local patterns, repeated structures, and spatial hierarchy. A suitable model must:

- **Preserve locality** — maintain spatial relationships between pixels
- **Reuse patterns** — apply the same pattern detector across the entire image
- **Use fewer parameters** — avoid the parameter explosion of fully connected layers

This motivates the introduction of **Convolutional Neural Networks (CNNs)**.

CNN is an optimized structure of ANN that helps extract features while preserving the spatial grid structure of images.

2 Images as Tensors

Before applying convolution, it is necessary to understand how images are represented mathematically. In CNNs, images are not vectors — they are multi-dimensional tensors.

2.1 What is a Pixel?

Pixel

A **pixel** is a numerical value representing image information at a specific location.

- **Grayscale image**: Each pixel has 1 intensity value.
- **RGB image**: Each pixel has 3 values — Red, Green, Blue.

A color image is therefore not 2D but **3D**, because each pixel carries depth information across color channels.

2.2 Image as a 3D Tensor

An RGB image is represented as:

$$(H, W, C)$$

where H = Height, W = Width, C = Channels (3 for RGB).

Example: A 32×32 RGB image $\rightarrow$ tensor of shape $(32, 32, 3)$.

The depth of an image is defined by the number of color channels. A grayscale image has only height and width (just one intensity value per pixel), so its depth is 1.

Creating an Image Tensor

```python
import torch

# Create a single RGB image
image = torch.randn(32, 32, 3)  # (H, W, C)
```

```
5
6 print("Image shape (H, W, C):", image.shape)
7 # Output: torch.Size([32, 32, 3])
```

Listing 2: Image tensor in standard format

2.2.1 Color Channels

An RGB image can be decomposed into three separate 2D matrices:

- **R channel** — Red intensity values
- **G channel** — Green intensity values
- **B channel** — Blue intensity values

RGB is used because it is a well-defined standard color model. With RGB, any color can be represented. Other color spaces exist as well — for example, **HSV** (Hue, Saturation, Value), which is used for HDR images. The choice of color channel depends on the application; converting between RGB and HSV is straightforward.

Grayscale vs RGB

While grayscale images can be used for classification, accuracy will typically be lower because RGB carries more color information. The goal is to mimic how the human visual system perceives images — with full color information.

2.3 PyTorch Tensor Convention

PyTorch uses a different ordering than the standard (H, W, C):

PyTorch format:

$$(N, C, H, W)$$

where:

- N = Batch size (number of images)
- C = Channels
- H = Height
- W = Width

Examples:

- Single RGB image: $(1, 3, 32, 32)$
- 10 RGB images: $(10, 3, 32, 32)$
- Single grayscale image: $(1, 1, H, W)$
- 20 RGB images of size 64×64: $(20, 3, 64, 64)$

PyTorch Tensor Convention

```
1 import torch
2
3 # Single image in PyTorch format
4 image_pt = torch.randn(1, 3, 32, 32)
5 print("PyTorch image shape (N, C, H, W):", image_pt.shape)
6 # Output: torch.Size([1, 3, 32, 32])
7
8 # Batch of 5 images
9 batch = torch.randn(5, 3, 32, 32)
10 print("Batch shape:", batch.shape)
```

```
# Output: torch.Size([5, 3, 32, 32])
```

Listing 3: Image tensors in PyTorch format

2.4 Why Channel Before Height and Width?

In convolution:

- Each filter operates across **all channels** simultaneously
- The channel dimension is treated as **depth**
- Convolution slides only across **height and width**

The filter must combine information from all channels (R, G, B) to detect meaningful patterns. However, the filter can only move spatially across height and width. Therefore, the channels are placed first so that the convolution operation can:

1. Differentiate between channels
2. Slide over each channel across height and width
3. Merge the information from all channels into a single output

Spatial movement of a filter happens across height and width only, not across depth. That is the fundamental reason for the (N, C, H, W) ordering.

Visualizing a Single Channel

```python
import torch
import matplotlib.pyplot as plt

# Create random image in PyTorch format
img = torch.randn(3, 32, 32)

# Visualize first channel
plt.imshow(img[0], cmap='gray')
plt.title("Channel 0")
plt.colorbar()
plt.show()

print("Tensor shape:", img.shape)
# Output: torch.Size([3, 32, 32])
```

Listing 4: Visualizing one channel of a random image

2.5 Role of the Batch Dimension

The batch dimension N serves several purposes:

- **Parallel processing**: Multiple images are processed simultaneously on the GPU
- **Memory management**: Instead of loading all images at once (which would overload RAM), images are divided into batches
- **Stable gradient updates**: Batch processing enables more stable gradient computation during training

Removing the batch dimension causes shape mismatch errors because PyTorch tensors require four parameters (N, C, H, W) for image convolution operations. Training pipelines become inconsistent without it.

3 Convolution — Intuition and Mechanics

Having identified the two problems with fully connected layers (too many parameters and loss of spatial structure), the key question is: **How can we process an image without flattening it?**

The answer is **convolution**. Convolution works locally and preserves spatial structure.

3.1 The Sliding Window Idea

Convolution Operation

Convolution is the operation of sliding a small window (called a **kernel** or **filter**) across an image, performing element-wise multiplication at each position, and summing the results to produce a single output value. This is analogous to the sliding window technique from data structures and algorithms, extended to 2D.

The process at each location:

1. Place the kernel on a local image patch
2. Multiply element-wise: each kernel value $\times$ corresponding image pixel
3. Sum all the products to produce **one number**
4. That number becomes a pixel in the output image
5. Slide the window to the next position and repeat

In DSA, the sliding window uses two pointers on a 1D array. For 2D images, this extends to four pointers defining a rectangular window that slides across height and width.

3.2 Convolution Example

Element-wise Convolution at One Location

```python
import torch

# Example 3x3 image patch
patch = torch.tensor([
    [1., 2., 1.],
    [0., 1., 0.],
    [2., 1., 2.]
])

# Example kernel (vertical edge detector)
kernel = torch.tensor([
    [1., 0., -1.],
    [1., 0., -1.],
    [1., 0., -1.]
])

result = (patch * kernel).sum()

print("Patch:\n", patch)
print("\nKernel:\n", kernel)
print("\nConvolution result at this location:", result.item())
# Output: 0.0
```

Listing 5: Manual convolution at a single position

The computation: $(1\times1)+(2\times0)+(1\times-1)+(0\times1)+(1\times0)+(0\times-1)+(2\times1)+(1\times0)+(2\times-1) = 1+0-1+0+0+0+2+0-2=0$.

A result of 0 means **no vertical edge detected** at this location.

3.3 Kernel as a Pattern Detector

Kernel / Filter

A **kernel** (also called a **filter**) is a small matrix of learnable weights that acts as a pattern detector. The kernel slides over the image and detects specific features at each location. Kernels and filters are alias names for the same thing.

The vertical edge detection kernel $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ works as follows:

- Left side gets **positive** weights
- Right side gets **negative** weights
- If there is a **strong intensity difference** between left and right (an edge), the output becomes large (positive or negative)
- If the region is uniform, the output is close to 0

Edge detection works by measuring contrast — the difference in intensity between adjacent regions. Where there is a boundary (e.g., between a dark and light region), the convolution output is large. Where the region is uniform, the output is small.

Filters as 2D Neurons

Filters can be thought of as **two-dimensional neurons**. In a standard ANN, a neuron is 1D and computes a weighted sum of all inputs. In a CNN, the kernel is a small 2D structure that slides over the image and computes local weighted sums. However, after feature extraction by convolution layers, fully connected (dense) layers are still used for final prediction.

3.4 Convolution with Multi-Channel Images

For an RGB image with 3 channels, the convolution operation works as follows:

1. The filter slides over **each channel separately** across height and width
2. The results from all three channels are **combined** (added or multiplied) into a single output matrix
3. This produces one feature map from the three-channel input

This is why the filter must process all channels but can only spatially move across height and width — the depth (channel) information is merged, not traversed spatially.

3.5 Output Size Reduction

Output size formula (without padding, stride = 1):

$$\text{Output size} = \text{Input size} - \text{Kernel size} + 1$$

Example: Input 5×5, Kernel 3×3:

$$5 - 3 + 1 = 3 \quad \Rightarrow \quad \text{Output is } 3 \times 3$$

The kernel cannot slide beyond the image boundaries, so the output is always smaller than the input (when no padding is used).

2D Convolution in PyTorch

```
import torch
import torch.nn.functional as F

```

```python
# Create a small 5x5 grayscale image
image = torch.randn(1, 1, 5, 5)

# Create 3x3 kernel
kernel = torch.randn(1, 1, 3, 3)

output = F.conv2d(image, kernel)

print("Input shape:", image.shape)     # [1, 1, 5, 5]
print("Kernel shape:", kernel.shape)   # [1, 1, 3, 3]
print("Output shape:", output.shape)   # [1, 1, 3, 3]
```

Listing 6: Using F.conv2d for 2D convolution

The convolution successfully reduced features from a 5×5 grid to a 3×3 grid while **maintaining the grid structure** — this is the key advantage over flattening.

3.6 Kernel Size Tradeoffs

Aspect	Small Kernels	Large Kernels
Output size	Larger (more features preserved)	Smaller (more features compressed)
Information loss	Less	More
Parameter count	More total parameters	Fewer total parameters
Preferred	Generally preferred	Situational use

Kernel size is part of the optimization process. There is no fixed kernel size — it must be chosen carefully. Smaller kernels are generally preferred because they preserve more spatial information, though they produce larger output feature maps that require more memory.

Since the kernel performs element-wise multiplication and summation, some information is lost in the process. Larger kernels compress more information into fewer values, leading to greater information loss.

3.7 Padding

When a kernel slides over an image without extending beyond boundaries, the output is smaller than the input. To **preserve the original image dimensions**, padding is used.

Padding

Padding adds extra pixels (typically with value 0) around the border of the input image so that the kernel can cover the full spatial extent. This ensures the output dimensions match the input dimensions.

Padding with zeros around the image allows the kernel to slide over edge and corner pixels, preserving the spatial resolution of the output.

4 Key Concepts from Deep Learning Context

4.1 Deep Learning and Data Requirements

Deep learning models require large datasets to train effectively. With small datasets, deep models (those with many layers) tend to **overfit** the training data.

An ANN with three hidden layers trained on the function $y = x^2$ with only 1,000 data points will overfit. Increasing the data points to 2,000 and using layers with sizes $1 \to 16 \to 32 \to 1$ (with learning rate 0.08) resolves the overfitting.

Data Requirements for Deep Learning

Sufficient data points are essential for deep learning models. On small datasets, deep networks will overfit. This is why deep learning is designed for large, heavy datasets — not small ones.

4.2 Curse of Dimensionality and Deep Learning

Traditional machine learning algorithms suffer from the curse of dimensionality. Deep learning models, particularly CNNs, are designed to handle high-dimensional data like images. However, if the data size is very small relative to the model complexity, overfitting can still occur.

Once spatial features are extracted using CNN through convolution, dimensionality reduction techniques like PCA can be applied in a hybrid model. The key is to first extract ordered spatial features via convolution, then apply any further transformations.

5 Summary

- **Fully connected networks fail for images** due to parameter explosion (a 32×32 RGB image already produces $\sim$300K parameters in one layer) and loss of spatial structure through flattening.
- **Images are 3D tensors** with shape (H, W, C). RGB images have 3 channels representing Red, Green, and Blue color intensities.
- **PyTorch uses** (N, C, H, W) **format** where N is batch size, C is channels, H is height, and W is width. Channels come before spatial dimensions because convolution slides spatially but processes all channels together.
- **Convolution** is a sliding window operation where a small kernel performs element-wise multiplication and summation at each position, producing a new feature map.
- **Kernels act as pattern detectors** — for example, a vertical edge detection kernel detects intensity differences between left and right regions.
- **Output size** follows Input $-$ Kernel $+ 1$ (without padding).
- **Padding** preserves input dimensions by adding zeros around the border.
- **CNN advantages over ANN**: preserves spatial locality, reuses patterns via shared kernels, and uses fewer parameters through local connectivity.